

baswe.Ai Engineer Accelerator™

Build These Six Projects

A step by step build guide for the six portfolio projects
that prove you can do AI engineering work

RAG Knowledge Graph RAG for Enterprise Data

AGENTS Multi-Agent Research Assistant

LLMOPS Self-Healing LLM Gateway

EVALS LLM-as-Judge with Human Calibration

FINE-TUNING Model Distillation Pipeline

SAFETY Automated Red-Team Harness

One skill · one project · one proof

baswe.ai

Before you start

Each of these six projects maps to one skill an AI engineering interview will test. Built together they cover the full surface: retrieval, agents, operations, evaluation, model economics, and safety. Built well, any one of them carries a whole interview loop on its own.

Read this page first, because it decides whether the next forty pages are worth your time.

Sequence them, do not sprint them

Build in the order they appear. Project 1 gives you a system worth evaluating. Project 4 evaluates it. Project 6 attacks it. Project 5 makes it cheaper. That dependency chain is deliberate, and a portfolio where the projects reference each other reads as an engineer with a system rather than a student with a checklist.

If you only have time for two, build Project 1 and Project 3. Retrieval plus infrastructure is the most common shape of a real AI engineering job.

What makes a project count

- ▶ A number. Every one of these projects produces a measured result: an accuracy delta, an availability figure, an agreement score, a break even volume. A project with no number is a tutorial you followed.
- ▶ A README that opens with the result. Benchmark table or demo video first, architecture second, setup instructions last. Nobody scrolls.
- ▶ A ninety second demo video. Most reviewers will never run your code. They will watch the video, read the top of the README, and decide.
- ▶ Honest failure notes. A section on what did not work and why is the single strongest credibility signal in a portfolio repository.
- ▶ Real data. Public filings, your own documents, open datasets. Toy data produces toy results and interviewers can tell instantly.

How each project is laid out

Why it signals. What this project proves that a tutorial does not, and the interview question it answers for you.

Five phases. The build, in order, with the specific decisions and the places people get stuck.

Done when. The finish line, so you stop building and start shipping.

Resume line and mistakes. How to describe it, and the four ways these projects usually go wrong.

Estimates assume ten to twelve hours a week. Halve them if this is full time work, and do not be discouraged if a phase takes twice as long as the estimate. Phase 1 always does.

Knowledge Graph RAG for Enterprise Data

Python · Neo4j · LangChain · Claude API · pgvector · FastAPI

Estimated build time: 3 to 4 weeks part time

Why this project signals

Almost every candidate has built vector RAG. Very few have built retrieval that answers questions requiring two or three hops across related entities. This project proves you understand where embeddings fail and what to do about it, which is exactly the follow up interviewers use to separate people who read about RAG from people who have debugged it.

PHASE 1

Extract entities and relationships into Neo4j

- ▶ Choose a corpus that actually has relationships. SEC filings, an internal wiki, research papers with citations, or product docs with dependency chains all work. A folder of unrelated blog posts has nothing to extract and the project dies here.
- ▶ Constrain the ontology before you write any code. Define five to ten entity types and eight to fifteen relationship types. An open ended extract all entities prompt produces a graph nobody can query.
- ▶ Chunk the documents, then run an extraction pass with Claude returning a strict schema: entity type, canonical name, relationship type, source chunk id, and confidence. Validate every response against the schema and retry the failures.
- ▶ Entity resolution is the hard part and the part that gets skipped. Acme Corp, Acme Corporation, and ACME must collapse into one node. Normalize, compare with embedding similarity above a tuned threshold, and store an alias list on the node.
- ▶ Write with MERGE rather than CREATE so re running ingestion is idempotent. Every edge carries source chunk id so you can cite the sentence that justified it.
- ▶ Budget the extraction cost per document before you run the full corpus, and cache results by document hash. This is where people accidentally spend two hundred dollars in an afternoon.

PHASE 2

Build the vector index alongside the graph

- ▶ Embed the same chunks into pgvector with metadata: document id, section path, date, and the entity ids that chunk mentions.
- ▶ Both stores share the same chunk ids. That shared key is the whole trick. It lets you cross from a graph path back to the original text, and from a retrieved passage into the graph neighborhood around it.
- ▶ Index with HNSW, tune ef_search, and measure recall at k on a small labeled set before moving on. Do not build the router on top of retrieval you have not measured.

PHASE 3

Route questions to the right retrieval path

- ▶ Write a cheap router: a small model call with a few shot prompt returning an enum, plus a low confidence fallback that runs both paths and merges.
- ▶ Route to the graph for connection questions, multi hop chains, comparisons across entities, and aggregations over relationships. Route to vectors for definitions, policy lookups, and single fact questions.
- ▶ On the graph path, extract entities from the question, resolve them to node ids, then run a parameterized Cypher query. Never let the model emit raw Cypher against your database. Keep a template library keyed by query type and let the model fill parameters only. This is a security control and it also makes results reproducible.
- ▶ Log every routing decision with the question and the outcome. You need this data for Phase 5 and you will not be able to reconstruct it later.

PHASE 4

Merge both sources into one grounded answer

- ▶ Graph traversal returns paths and vector search returns passages. Convert paths into readable statements before they reach the prompt, because raw triples generate awkward text.
- ▶ Deduplicate across the two sets, then assemble context with explicit labels separating graph derived facts from retrieved passages.
- ▶ Require a citation per claim, then validate that every citation resolves to a chunk id that was actually retrieved. Reject and regenerate when it does not. This costs nothing and eliminates invented sources.

PHASE 5

Benchmark against plain vector RAG and publish the delta

- ▶ Build a question set of fifty to a hundred items, stratified by difficulty: single hop, two hop, three hop, aggregation, and out of scope questions the system should refuse.
- ▶ Run both systems on the same set and report accuracy broken out by hop count. The expected shape is parity on single hop questions and a widening gap as hops increase. That curve is the story.
- ▶ Report latency and cost per query for both, plus the one time ingestion cost of the graph. Being honest that graph RAG is slower and more expensive to build is what makes the accuracy claim credible.
- ▶ This benchmark table is the portfolio artifact. Put it at the top of the README, above the architecture diagram.

Done when

A FastAPI endpoint answers questions with validated citations, and the README opens with a benchmark table comparing your system to a vector only baseline by hop count.

Resume line: Built a hybrid knowledge graph and vector RAG system over N documents; raised

multi hop question accuracy from X to Y percent against a vector baseline at Z ms p95.

Where this usually goes wrong

- ▶ An open ended ontology, which produces a graph you cannot write queries against.
- ▶ Skipping entity resolution, so the same company exists as four disconnected nodes.
- ▶ Letting the model write raw Cypher, which is both a security hole and unreproducible.
- ▶ Choosing a corpus with no real relationships, which makes the graph pointless.

Stretch goals: Incremental graph updates instead of full reingestion, community detection for corpus level summaries, and a graph visualization in the response UI.

Multi-Agent Research Assistant

Python · LangGraph · Claude API · Tavily · Redis · FastAPI

Estimated build time: 3 to 4 weeks part time

Why this project signals

Interviewers have seen hundreds of agent demos and almost none that survive a restart, cap their own spend, or explain what they did. The differentiator here is not the agents. It is durable state, budgets, and step level tracing, which are the three things a production agent has and a tutorial agent does not.

PHASE 1

Build three specialists

- ▶ Planner takes a research question and returns a structured plan of sub questions. Researcher executes one sub question with search and returns findings. Writer turns findings into a report with citations.
- ▶ A specialist is a prompt plus a tool set plus an output schema. It is not a personality. Skip the you are a world class researcher framing and define Pydantic models for every hand off instead.
- ▶ Test each agent alone before wiring them together. If the planner produces vague sub questions in isolation, no orchestration will rescue it.

PHASE 2

Add a supervisor that routes and reviews

- ▶ Model the system as a LangGraph state graph. Nodes are agents, edges are supervisor routing decisions, and the shared state object is the single source of truth.
- ▶ After each researcher pass the supervisor checks the output against the plan: is this sub question answered, are sources missing, is another pass justified.
- ▶ Enforce budgets in the graph rather than in prompts: maximum sub questions, maximum searches per sub question, maximum total tokens, and a wall clock ceiling. A prompt asking the model to be efficient is not a budget.
- ▶ Allow the reviewer to send work back exactly once. Unlimited revision loops are how agent runs quietly cost twelve dollars.

PHASE 3

Give the researcher live search with source tracking

- ▶ Every finding is a record: claim, source URL, snippet, and retrieval timestamp. Findings without provenance are unusable in the writer step.

- ▶ Fetch and extract the page rather than trusting search snippets, which are truncated and often misleading.
- ▶ Deduplicate by URL and cap results per domain so a single site cannot dominate a report.
- ▶ Handle each failure path explicitly: no results, rate limited, paywalled, timed out. Each returns a structured status the supervisor can act on rather than an exception that kills the run.

PHASE 4

Persist run state in Redis

- ▶ Write graph state after every node: run id, plan, completed sub questions, findings so far, and cumulative token spend.
- ▶ Now a crashed or redeployed process resumes instead of replaying, which is the difference between a demo and a service.
- ▶ The same store gives you idempotency, since re entering a completed node returns the cached result, and it powers a status endpoint so callers can see progress.

PHASE 5

Ship a report endpoint with citations and a trace

- ▶ POST /research returns a run id immediately. GET /research/{id} returns status and then the finished report. Stream progress events so the interface is not a four minute spinner.
- ▶ Every claim in the report carries a citation, and each citation is validated against a real finding record before the report is returned.
- ▶ Expose GET /research/{id}/trace returning the full step log: which agent ran, what it received, what it produced, which tools it called, tokens, and latency per step.
- ▶ That trace endpoint is what makes this project interview proof. When they ask how you would debug an agent that returned a bad answer, you open the trace instead of describing one.

Done when

A research question returns a fully cited report in under five minutes, a killed process resumes from where it stopped, and every run has a viewable step by step trace.

Resume line: Built a multi agent research system with supervisor and worker topology, durable run state in Redis, per run cost caps, and step level tracing for debugging.

Where this usually goes wrong

- ▶ Agents defined by personality prompts instead of typed schemas, which makes hand offs unreliable.
- ▶ No budget enforcement, so a single run silently costs more than the whole project.
- ▶ Treating search failures as exceptions rather than states the supervisor can route around.
- ▶ No trace, which means you cannot answer the one debugging question every interviewer asks.



PROJECT 3 · LLMOPS

Self-Healing LLM Gateway

Python · LiteLLM · FastAPI · Redis · Prometheus · Grafana

Estimated build time: 2 to 3 weeks part time

Why this project signals

This is the project that reads as infrastructure rather than experimentation, which is exactly the transition you are trying to signal. It also gives you the strongest demo of the six, because you can break a provider live and show traffic rerouting on a dashboard while you talk.

PHASE 1

Route every model call through one service

- ▶ Build a FastAPI service that accepts an OpenAI compatible request shape so existing clients need almost no change to adopt it.
- ▶ Use LiteLLM to normalize provider differences. Configure at least three providers so failover has somewhere to go.
- ▶ Require metadata on every request: tenant, feature, and request id. Without these you can never attribute cost, and cost attribution is half the value of a gateway.

PHASE 2

Track health per provider

- ▶ Maintain a rolling window per provider covering success rate, p50, p95, and p99 latency, and a proper error taxonomy separating rate limits, timeouts, server errors, content filters, and auth failures.
- ▶ Keep counters in Redis using a sliding window so restarts do not reset your view of provider health.
- ▶ Expose a /metrics endpoint for Prometheus. Instrument before you build failover, because failover logic you cannot observe is guesswork.

PHASE 3

Fail over automatically

- ▶ Implement a circuit breaker per provider with closed, open, and half open states. Trip on error rate above threshold within the window, or p95 latency above budget.
- ▶ When a breaker opens, route to the next provider in the preference list for that request class. Preference lists differ by class, since a cheap classification call and a long form generation call should not fail over the same way.
- ▶ Half open probes are what make it self healing. Send a small percentage of traffic back to a recovering provider, close the breaker if it succeeds, reopen immediately if it does not.

- ▶ For latency sensitive classes, add hedged requests: fire a second provider after N milliseconds, take whichever returns first, cancel the loser. Document the cost trade off, because hedging roughly doubles spend on hedged calls.

PHASE 4

Queue and retry deferrable work

- ▶ Classify requests as interactive or deferrable at the API boundary. Interactive calls fail fast, deferrable calls survive an outage.
- ▶ Push deferrable work to a Redis queue with exponential backoff and jitter when every provider is degraded, instead of returning an error to the caller.
- ▶ Attach idempotency keys so a retry cannot duplicate a side effect. This is the detail that signals you have operated something in production.

PHASE 5

Put it on a Grafana board and then break it on purpose

- ▶ Panels worth building: requests per second by provider, error rate, p95 latency, circuit state timeline, failover events, queue depth, and cost per hour by tenant and by feature.
- ▶ Add a chaos endpoint or proxy that injects errors and latency into one provider on demand.
- ▶ Record a ninety second screen capture: healthy traffic, a provider degraded, the breaker tripping, traffic rerouting, and the breaker closing again as it recovers.
- ▶ That video goes at the top of the README. It communicates more in ninety seconds than a page of architecture description.

Done when

You can degrade a provider live while traffic keeps flowing, and the dashboard shows the trip, the reroute, and the automatic recovery.

Resume line: Built an LLM gateway with per provider circuit breaking and automatic failover; sustained X percent availability through simulated provider outages with per tenant cost attribution.

Where this usually goes wrong

- ▶ No cost attribution, which removes the main reason companies build gateways.
- ▶ Retrying calls that are not idempotent, which turns a blip into duplicated side effects.
- ▶ No half open state, so the breaker opens once and never heals.
- ▶ A dashboard with no chaos test, which leaves the failover logic unproven.

Stretch goals: Add a semantic cache layer, a model tiering router that sends easy requests to a small model, per tenant rate limits and budgets, and request logging with PII redaction.

LLM-as-Judge with Human Calibration

Python · DeepEval · Claude API · Postgres · Streamlit

Estimated build time: 2 to 3 weeks part time

Why this project signals

Evaluation is the round candidates most often say they wish they had prepared for, and almost nobody has quantified whether their judge agrees with a human. Producing a kappa score and three measured bias numbers puts you ahead of engineers with more years of experience than you.

PHASE 1

Write judge prompts against a real rubric

- ▶ Pick one task with real outputs. RAG answers from Project 1 work well and make your portfolio cohere.
- ▶ Define three to five criteria, each on a three or five point ordinal scale with every point defined in words. A one to ten scale produces noise you will spend a week chasing.
- ▶ Have the judge return structured output with its reasoning first and scores second. Reason before score measurably improves consistency, and you should measure it rather than assume it.

PHASE 2

Hand label 200 outputs to build the gold standard

- ▶ Sample stratified across easy, hard, adversarial, and deliberately broken outputs. A calibration set with no failures has no variance and teaches the judge nothing.
- ▶ Build a small Streamlit labeling interface showing input, output, and rubric, capturing a score per criterion plus a free text note. Store results in Postgres with labeler id and timestamp.
- ▶ Label twenty of them twice on different days and compute your own self agreement. That number is your ceiling, because the judge cannot be more consistent than the humans defining correct.

PHASE 3

Measure agreement, then close the gaps

- ▶ Compute weighted Cohen kappa for the ordinal scores and Spearman correlation alongside it. Report both.
- ▶ Pull the twenty largest disagreements and read them one by one. In nearly every case the rubric is ambiguous rather than the model being wrong, which is the single most useful lesson in this project.
- ▶ Sharpen the wording, add two or three anchor examples per score point, re run, and re measure. Track agreement across iterations and chart it. That improvement curve belongs in the README.

PHASE 4

Test the judge for its known biases

- ▶ Position bias: run pairwise comparisons in both orders and measure how often the verdict flips. A flip rate above roughly ten percent means your pairwise setup is not trustworthy yet.
- ▶ Length bias: hold quality constant and correlate score against output length. If longer wins, your rubric is rewarding volume.
- ▶ Self preference: have one model family judge outputs from itself and a competitor, then swap the judge and compare verdicts on the same pairs.
- ▶ Publish all three numbers. Most working engineers have never quantified any of them, which is precisely why it stands out in an interview.

PHASE 5

Gate releases on calibrated scores

- ▶ Wire the judge into DeepEval and run it as a CI job against a fixed evaluation set on every pull request.
- ▶ Set explicit thresholds: no criterion may drop more than a defined amount, and the safety criterion may not drop at all. Fail the build on regression.
- ▶ Store every run in Postgres and chart scores over time in Streamlit, so quality has a trend line rather than an anecdote.
- ▶ Demo it by opening a pull request that intentionally weakens a prompt and letting CI reject it on camera.

Done when

A pull request that degrades output quality fails CI automatically, and the README reports judge to human agreement plus three measured bias numbers.

Resume line: Built an LLM as judge evaluation pipeline calibrated against N human labels at kappa X; quantified position, length, and self preference bias and gated releases on it in CI.

Where this usually goes wrong

- ▶ No human baseline at all, which makes every judge score unfalsifiable.
- ▶ One to ten scales, which produce inconsistent scores across runs and labelers.
- ▶ Judging outputs with the same model that produced them.
- ▶ Tuning the rubric on the same examples you report agreement on, which is leakage.

Stretch goals: Active learning to select the next most informative examples to label, score slicing by user cohort, and drift detection on sampled production traffic.

Model Distillation Pipeline

Python · Hugging Face · PyTorch · Claude API · W&B · vLLM

Estimated build time: 4 to 5 weeks part time

Why this project signals

This is the project that lets you talk about unit economics, which is the fastest way to sound senior. The output is not a model, it is a break even analysis: the request volume above which owning the model beats renting it. Very few candidates can produce that number from their own work.

PHASE 1

Pick one narrow task a frontier model already does well

- ▶ Strong candidates: structured extraction from one document type, classification across many labels, query rewriting, SQL generation against one schema, or PII redaction.
- ▶ Weak candidates: open ended chat and anything requiring broad world knowledge. You are transferring a capability, not creating one.
- ▶ Establish the teacher accuracy first on a held out set. That number is your ceiling and your entire comparison depends on it being honest.

PHASE 2

Generate a training set from the teacher's best outputs

- ▶ Run the teacher over real inputs rather than synthetic prompts, so the training distribution matches the traffic the student will actually see.
- ▶ Filter for quality: validate every output against the schema, drop failures rather than teaching them, and deduplicate near identical inputs.
- ▶ Aim for five to ten thousand examples to start. Split train, validation, and test before any tuning and do not touch the test set until Phase 4.
- ▶ This generation run is the main cash cost of the project. Estimate it before you launch it and put the actual number in the README, because cost transparency is part of what you are demonstrating.

PHASE 3

Fine tune an 8B student

- ▶ Use a seven to eight billion parameter open model with LoRA, or QLoRA if you want it to fit on a single smaller rented GPU.
- ▶ Log everything to Weights and Biases: loss curves, learning rate schedule, validation metric per epoch, and sample outputs at each checkpoint.

- ▶ Watch for overfitting after two or three epochs on a small dataset, and for degradation if the learning rate is too aggressive.
- ▶ Train at least two configurations, for example LoRA rank eight against rank thirty two, so you have a real trade off to discuss rather than a single lucky run.

PHASE 4

Benchmark student against teacher on three axes

- ▶ Quality on the untouched test set using task metrics, plus the judge from Project 4 where the output is open ended.
- ▶ Cost per thousand requests: teacher API pricing against student GPU hour cost amortized across measured throughput, including idle time.
- ▶ Latency at p50 and p95 for both, measured under comparable concurrency.
- ▶ Present it as one table. A typical honest result is the student reaching the mid nineties percent of teacher quality at a fraction of the cost. If yours underperforms, publish that and explain why, because a reported negative result reads as more trustworthy than a suspiciously perfect one.

PHASE 5

Serve it with vLLM and publish the savings

- ▶ Serve with vLLM for continuous batching and paged attention. Measure throughput as you raise concurrency and find the point where latency starts degrading.
- ▶ Compute the break even volume: below X requests per day the API is cheaper, above it the self hosted student wins. This single number is the most senior artifact in the entire portfolio.
- ▶ Ship a router in front of it: the student handles the task and escalates to the teacher on low confidence or schema validation failure. Report the escalation rate.

Done when

A three axis benchmark table, a stated break even request volume, and a served endpoint with a working escalation path back to the teacher.

Resume line: Distilled a frontier model task into an 8B student at X percent of teacher accuracy and one twentieth the cost; served with vLLM with break even at Y requests per day.

Where this usually goes wrong

- ▶ Distilling a task the teacher performs poorly, which caps the student below usable.
- ▶ No untouched test set, so every quality claim is contaminated.
- ▶ Comparing costs without counting idle GPU time, which flatters the self hosted option.
- ▶ No fallback to the teacher, which makes the system fragile on the tail of the distribution.

Stretch goals: Quantize to four bit and remeasure the three axes, collect production data continuously for retraining, and train per customer adapters swapped at serving time.



PROJECT 6 · SAFETY

Automated Red-Team Harness

Python · garak · PyRIT · Claude API · FastAPI · Postgres

Estimated build time: 3 to 4 weeks part time

Why this project signals

Safety is moving from a specialist concern to a standard interview topic, and almost nobody has a working artifact to point at. This project also ties the portfolio together, because the systems it attacks are your own earlier projects.

Scope note to put in your README: this harness runs against applications you own and control. Point it at your own Project 1 and Project 2 builds, never at third party systems.

PHASE 1

Build the attack battery

- ▶ Cover the categories that matter: direct jailbreaks and persona attacks, indirect injection through documents your app retrieves, system prompt extraction, PII extraction, harmful content relevant to your domain, tool abuse and privilege escalation, and encoding tricks such as base64 or unusual unicode.
- ▶ Start from garak probes and PyRIT datasets for breadth, then write your own domain specific cases. That domain layer is what makes this your project rather than a wrapper around someone else's tool.
- ▶ Store each attack as a row: id, category, severity, payload, the safe behavior you expect, and the source it came from.

PHASE 2

Run it against your own AI app on a schedule

- ▶ Define a target adapter interface, a function taking a payload and returning a response plus metadata, so the harness can point at any application you build.
- ▶ Aim it at your Project 1 RAG service first, since retrieval systems are where indirect injection actually bites. Plant a poisoned document in the corpus and see whether the pipeline follows its instructions.
- ▶ Wrap it in a FastAPI service with scheduled runs plus a CI hook so every deploy triggers a suite run.
- ▶ Parallelize with a concurrency cap and proper rate limit handling, or your own harness will be the thing that takes the app down.

PHASE 3

Score every response for violations

- ▶ Run deterministic checks first because they are free and unambiguous: did the system prompt leak verbatim, did a PII pattern appear, did a blocked tool get invoked, did an outbound URL carry data.
- ▶ Then run a classifier or judge call for policy violations that need interpretation.
- ▶ Use three outcomes rather than two: pass, fail, and needs review. The review queue is where you learn what your classifier is missing.
- ▶ Store the full request and response with model version and application version, so results stay comparable as both change.

PHASE 4

File findings automatically with severity and repro

- ▶ Score severity as exploitability against impact, mapped to critical, high, medium, and low.
- ▶ On a confirmed failure, generate a finding containing a minimal reproduction, the exact payload, the response received, and a suggested mitigation, then open a GitHub issue through the API.
- ▶ Deduplicate against open findings using attack id plus a response signature, or your first full run will file forty copies of the same issue and nobody will look at any of them.

PHASE 5

Re run after each fix to prove the hole stays closed

- ▶ Every confirmed finding becomes a permanent regression case in the suite. Nothing is ever removed after it is fixed.
- ▶ Build a dashboard showing findings by severity over time, mean time to close, pass rate by attack category, and a regression count that must stay at zero.
- ▶ The demo is the closed loop: a vulnerability found, a fix shipped, and the next scheduled run confirming it stays closed. Record it, because that loop is the entire point of the project.

Done when

Scheduled runs against your own applications, findings filed automatically with reproduction steps, and a dashboard trend line showing pass rate improving with zero regressions.

Resume line: Built an automated red team harness covering N attack categories against internal AI services; integrated into CI, auto files findings with reproductions, and enforces zero regressions on closed vulnerabilities.

Where this usually goes wrong

- ▶ Running it once instead of continuously, which turns it into a report rather than a control.
- ▶ Binary pass and fail with no review queue, which hides everything your scorer is uncertain about.

- ▶ No deduplication, which floods the issue tracker and trains everyone to ignore it.
- ▶ Pointing it at systems you do not own, which is out of scope for this project and for your career.

Stretch goals: Adaptive attacks that mutate after a refusal, multi turn attack chains that build context across messages, and coverage tracking against a published taxonomy such as the OWASP LLM Top 10.

Packaging the work

A finished project that nobody can evaluate in two minutes is an unfinished project. Every repository gets the same five things, in this order.

- ▶ One line at the top saying what it does and what it achieved, with the number in it.
- ▶ The result immediately after: a benchmark table, a dashboard screenshot, or an embedded demo video.
- ▶ An architecture diagram, one image, showing the data path rather than every class.
- ▶ A short design decisions section covering the two or three real trade offs you made and what you gave up.
- ▶ A what did not work section. This is the part experienced engineers read first and the part nobody writes.

Talking about them in the interview

Lead with the problem and the number, not the stack. Not I used LangGraph and Redis, but the runs were losing four minutes of work on every deploy, so I made the state durable and added a cost ceiling after one run cost eleven dollars. The second version invites the follow up questions you have already prepared for.

Then have one failure ready per project. Every interviewer asks what went wrong, and the candidates who have a specific answer with a specific fix are the ones who get the offer.

Six projects. **One portfolio that answers the question.**

Nobody gets hired for knowing what RAG is. They get hired because someone can see the thing they built and the number it produced.

skool.com/baswe-ai

baswe.Ai Engineer Accelerator™